Entwicklerdokumentation

Nils Generlich, Yannik-Joel Herrchen, Nico Kroll, Paul Marx, Richard Schultz, Marlene Winter

28. Juni 2024

Inhaltsverzeichnis

1. Architecture Notebook: Modell eines automatisierten Fabrikbetriebes - Frau Holle (I04) 1
1.1. Zweck 1
1.2. Architekturziele und Philosophie 1
1.3. Annahmen und Abhängigkeiten 1
1.4. Architektur-relevante Anforderungen 2
1.5. Entscheidungen, Nebenbedingungen und Begründungen 2
1.6. Architekturmechanismen 3
1.7. Wesentliche Abstraktionen 4
1.8. Schichten oder Architektur-Framework 5
1.9. Architektursichten (Views) 5
2. Softwaredokumentation 11
2.1. Allgemeine Informationen 11
2.2. Frontend 11
2.3. Backend 18
2.4. Datenbank 34

1. Architecture Notebook: Modell eines automatisierten Fabrikbetriebes - Frau Holle (I04)

1.1. Zweck

Dieses Dokument beschreibt die Philosophie, Entscheidungen, Nebenbedingungen, Begründungen, wesentliche Elemente und andere übergreifende Aspekte des Systems, die Einfluss auf Entwurf und Implementierung haben.

1.2. Architekturziele und Philosophie

Das System ist eine Datenbankanwendung für die Visualisierung und Verwaltung von Prozessschritten einer Halbleiterfabrik. Diese soll für mehrere Jahre nutzbar sein. Außerdem ist die Weiterentwicklung dieses Systems durch zukünftige Gruppen geplant. Dadurch soll die Architektur darauf abzielen, eine **langlebige, erweiterbare und robuste Datenbankanwendung** zu bieten, welches auch bei Abstürzen resistent gegenüber Datenverlust ist.

Für die intuitive Bedienung der Anwendung sollen die Prozesse und Datenausgabe übersichtlich und verständlich dargstellt werden. Hierzu sind **einfache Bedienelemente** zu verwenden, welche die Möglichkeiten für Fehlentscheidungen der Nutzer minimieren. Die Anwendung soll in verschiedenen Browsern nutzbar sein.

Da die Anwendung auf einem RaspberryPi betrieben wird, ist die Architektur darauf ausgelegt die Ressourcen des RaspberryPi möglichst effizient zu nutzen. Es sollen größere Mengen an Daten verarbeitet werden können, ohne dass die Anwendung abstürzt oder die Performance stark beeinträchtigt wird. So ist eine gut skalierbare Datenbankanwendung zu entwickeln.

1.3. Annahmen und Abhängigkeiten

Annahmen

- Nutzer des Systems nutzen moderne Browser (Chrome, Firefox, Edge, Safari), welche den HTML5-Standard unterstützen
- Die von uns genutzten Bibliotheken und Frameworks sind so gut wie fehlerfrei und werden zukünftig weiterentwickelt und gepflegt

Abhängigkeiten

- Wir sind darauf angewiesen, dass die von uns genutzten Bibliotheken und Frameworks weiterhin gepflegt werden.
- Wir sind auf die einwandfreie Funktion des Raspberry Pi angewiesen

1.4. Architektur-relevante Anforderungen

Funktional

- **SWFA-1:** Das System muss Zugriffsbeschränkungen gewährleisten, da nicht jeder Anwender jeden Use Case ausführen darf. Diese Zugriffsbeschränkungen erlauben es dem Anwender nur für ihn vorgesehene Funktionen auszuführen und nur auf für ihn zugängliche Daten zuzugreifen.
- **SWFA-2:** Das System muss zeitgesteuerte Aktivitäten in Form eines Timers ermöglichen, da Statusabfragen der Lose und Maschinen ausgewertet werden müssen.
- **SWFA-3:** Das System muss Eingaben des Anwenders und Statusupdates persistent speichern können.
- **SWFA-4:** Das System muss eine Config-Datei bereitstellen, in der systemweite Informationen eingestellt werden können.

Benutzbarkeit (Usability)

- **QAGU-1:** Die Gesamtheit der Interaktionselemente soll minimiert werden.

Zuverlässigkeit

- **QAGZ-1:** Das System muss bei einem Netzwerkausfall die Konsitenz der Daten gewährleisten

Wartbarkeit (Supportability)

- **QAGW-1:** Zur besseren Wartbarkeit und zum besseren Verständnis bei auftretenden Fehlern, muss das Entwicklerteam eine umfangreiche Entwicklerdokumentation bereitstellen.

Zusätzliche Anforderungen

- **Ressourceneinschränkungen:** Das System soll auf einem RaspberryPi lauffähig sein, wodurch die Ressourcen des Systems begrenzt sind.

1.5. Entscheidungen, Nebenbedingungen und Begründungen

1. Wir nutzen eine relationale Datenbank mit dem Datenbankmanagementsystem **PostgreSQL**, da die Nutzung von PostgreSQL uns vorgegeben wurde und dieses bekannt für seine Zuverlässigkeit und Skalierbarkeit ist.
2. Wir implementieren die Anwendung auf einem **Raspberry Pi**, da uns dieser vorgegeben wurde.
3. Als Betriebssystem nutzen wir **Raspberry Pi OS**, da dieses Betriebssystem für den Raspberry Pi optimal angepasst ist und eine einfache Bedienung ermöglicht.
4. Wir nutzen **Python** als Programmiersprache für die Middleware, da uns diese vorgegeben wurde und diese eine einfache und schnelle Entwicklung ermöglicht.
5. Wir benutzen **SQL-Alchemy** als Framework, da diese Bibliothek eine intuitive und effiziente Interaktion mit der PostgreSQL-Datenbank ermöglicht und vorraussichtlich auch in Zukunft

weiterentwickelt wird.

6. Wir nutzen **Flask** als Web-Framework, da dieses ein recht leichtgewichtiges Framework ist, welches wenige Resourcen benötigt. Außerdem ist Flask einfach zu erlernen, da es aufgrund seiner aktiven Entwickler-Community viele Tutorials und Hilfestellungen gibt.
7. Als Schnittstelle zwischen Middleware und Web-GUI nutzen wir **RESTful API-Endpunkte**, da diese eine standardisierte und effiziente Kommunikation zwischen den beiden Systemteilen ermöglichen.
8. Wir nutzen **HTML und CSS** für die Benutzeroberfläche, da diese Technologien sehr gut mit Flask integrierbar sind. Diese Technologien ermöglichen die Entwicklung einer ansprechenden und interaktiven Benutzeroberfläche für die Anwendung. Zur Verbesserung der Interaktivität kann auch JavaScript genutzt werden.
9. Für die Generierung von dynamischen Inhalten in der Webanwendung benutzen wir die Template-Engine **Jinja2**. Diese ist weit verbreitet und ist direkt in Flask integriert.

1.6. Architekturmechanismen

Doku "Concept: Architectural Mechanism"

1.6.1. Persistenz (Persistence)

- **Zweck:** Lesen und Schreiben von gespeicherten Daten.
- **Eigenschaften:** Unterstützt die dauerhafte Speicherung von Daten.
- **Funktion:** Verwendung eines relationalen DBMS (PostgreSQL) zur Datenpersistenz.
- **Ausschnit Zugehöriger SWFA:** SWFA-3, QAGZ-1

1.6.2. Sicherheit (Security)

- **Zweck:** Schutz des Zugriffs auf Ressourcen oder Informationen.
- **Eigenschaften:** Beinhaltet Authentifizierung, Autorisierung und Datenverschlüsselung.
- **Funktion:** Gewährleistung der Datensicherheit und des beschränkten Zugriffs basierend auf Benutzerrollen.
- **Ausschnit Zugehöriger SWFA:** SWFA-1

1.6.3. Transaktionsmanagement (Transaction Management)

- **Zweck:** Handhabung von ACID-Transaktionen.
- **Eigenschaften:** Unterstützt die Integrität und Zuverlässigkeit von Datenbankoperationen.
- **Funktion:** PostgreSQL bietet native Unterstützung für Transaktionsmanagement, um Konsistenz und Zuverlässigkeit der Daten zu gewährleisten.
- **Ausschnit Zugehöriger SWFA:** SWFA-3, QAGZ-1

1.6.4. Mega-Daten (Mega-data)

- **Zweck:** Unterstützung für die Verarbeitung sehr großer Datenmengen.
- **Eigenschaften:** Skalierbare Datenverarbeitung und -speicherung.
- **Funktion:** PostgreSQL und Python-Middleware für effiziente Handhabung und Verarbeitung großer Datensätze.
- **Ausschnit Zugehöriger SWFA:** Ressourceneinschränkungen

1.6.5. Terminierung (Scheduling)

- **Zweck:** Ermöglicht die Planung und Ausführung von Aufgaben zu bestimmten Zeiten oder unter bestimmten Bedingungen.
- **Eigenschaften:** Unterstützt zeitgesteuerte und ereignisbasierte Aufgaben. Flexibel einsetzbar für regelmäßige oder einmalige Ereignisse.
- **Funktion:** Im System ermöglicht dies die Automatisierung von Datenerfassung und die Gewährleistung des Fertigungsflusses, für zeitgesteuerte Datenbank-Updates.
- **Ausschnit Zugehöriger SWFA:** SWFA-2

1.6.6. Fehlermanagement (Error Management)

- **Zweck:** Erkennung, Weiterleitung und Berichterstattung von Fehlern.
- **Eigenschaften:** Robuste Fehlerbehandlung und klare Benachrichtigungssysteme.
- **Funktion:** Implementierung eines Systems, das Fehler effektiv erkennt und dem Benutzer verständliche Meldungen liefert.
- **Ausschnit Zugehöriger SWFA:** QAGZ-1

Diese Architekturmechanismen sind essentiell für die Entwicklung einer robusten, sicheren und skalierbaren Datenbankanwendung, die auf einem RaspberryPi mit Raspbian Betriebssystem betrieben wird.

1.7. Wesentliche Abstraktionen

- **Los**: enthält alle Informationen zum Los
- **Tool**: enthält alle Informationen zu den Maschienen
- **Prodflow**: enthält die n-zu-m-Verbindung von Workflow zu Los
- **Workflow**: enthält alle Informationen zu den Teilplänen (einzelnen Fertigungschritte)
- **PTime**: enthält alle Informationen zu Prozesszeiten
- **Entities**: enthält alle Informationen zu den MES Entitäten
- **User**: enthält alle Fremdschlüssel zu Nutzerinformationen
- **User_Information**: enthält alle Information, die keine Adressdaten sind
- **Config**: enthält alle Informationen zu den Konfigurationen

1.8. Schichten oder Architektur-Framework

Die Architektur basiert auf einem mehrschichtigen Framework, das Folgendes umfasst:

1.8.1. Datenbankebene (PostgreSQL)

- **Verantwortlich für die Speicherung und das Management der Daten** mit Fokus auf Effizienz und Skalierbarkeit für den Einsatz auf einem Raspberry Pi.

1.8.2. Middleware-Ebene (Python)

- **Python-Anwendung:** Dient als zentrale Vermittlungsstelle zwischen Datenbank und Benutzeroberfläche.
- **Datenbankzugriffsschicht:** Nutzt die SQLAlchemy-Bibliothek für den Zugriff auf die PostgreSQL-Datenbank.
- **Logikschicht:** Beinhaltet die Geschäftslogik, Datenvalidierung und -aufbereitung.

1.8.3. Benutzeroberflächenebene (Flask-GUI)

- **Flask als Web-Framework:** Vereinfacht die Erstellung und Verwaltung von Webanwendungen.
- **User Interface:** UI mittels HTML-Templates, CSS-Stylesheets und gegebenenfalls JavaScript-Code für die Interaktion mit dem Endbenutzer.

1.8.4. Schnittstellen

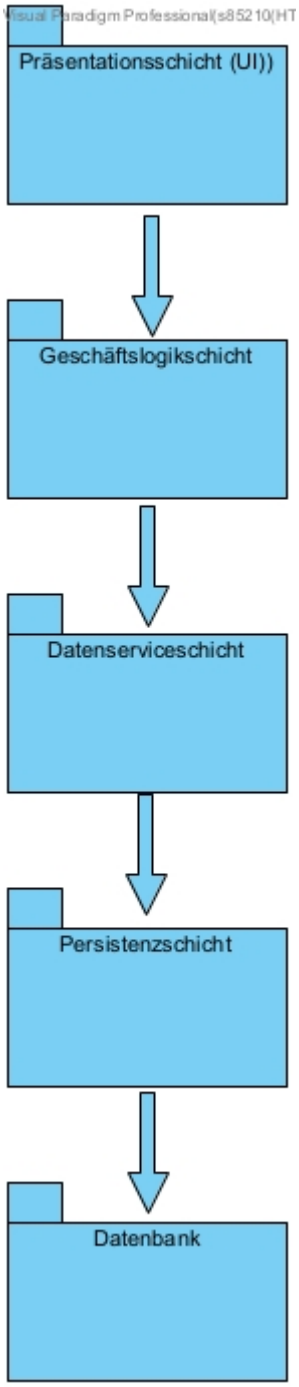
- **Datenbank-Middleware-Schnittstelle:** Sicherer Datenaustausch über die ORM-Schicht (Object-Relational Mapping) von SQLAlchemy.
- **Middleware-GUI-Schnittstelle:**
    - RESTful API-Endpunkte, die mit einer bestimmten HTTP-Methoden verbunden sind um entsprechende Middleware-Aktionen aufzurufen
    - Jinja2 als Template-Engine, mit welcher dynamische Inhalte auf HTML-Webseite abgebildet werden können

Diese Architektur fördert die langfristige Nutzbarkeit und Weiterentwicklung der Anwendung, selbst auf einem Raspberry Pi mit begrenzten Ressourcen.

1.9. Architektursichten (Views)

Folgende Sichten stellen die grundlegende Architektur des Frau Holle Systems dar:

1.9.1. Logische Sicht



Folgende Sichten werden entsprechend des **C4-Modells** für die Architekturbeschreibung verwendet:

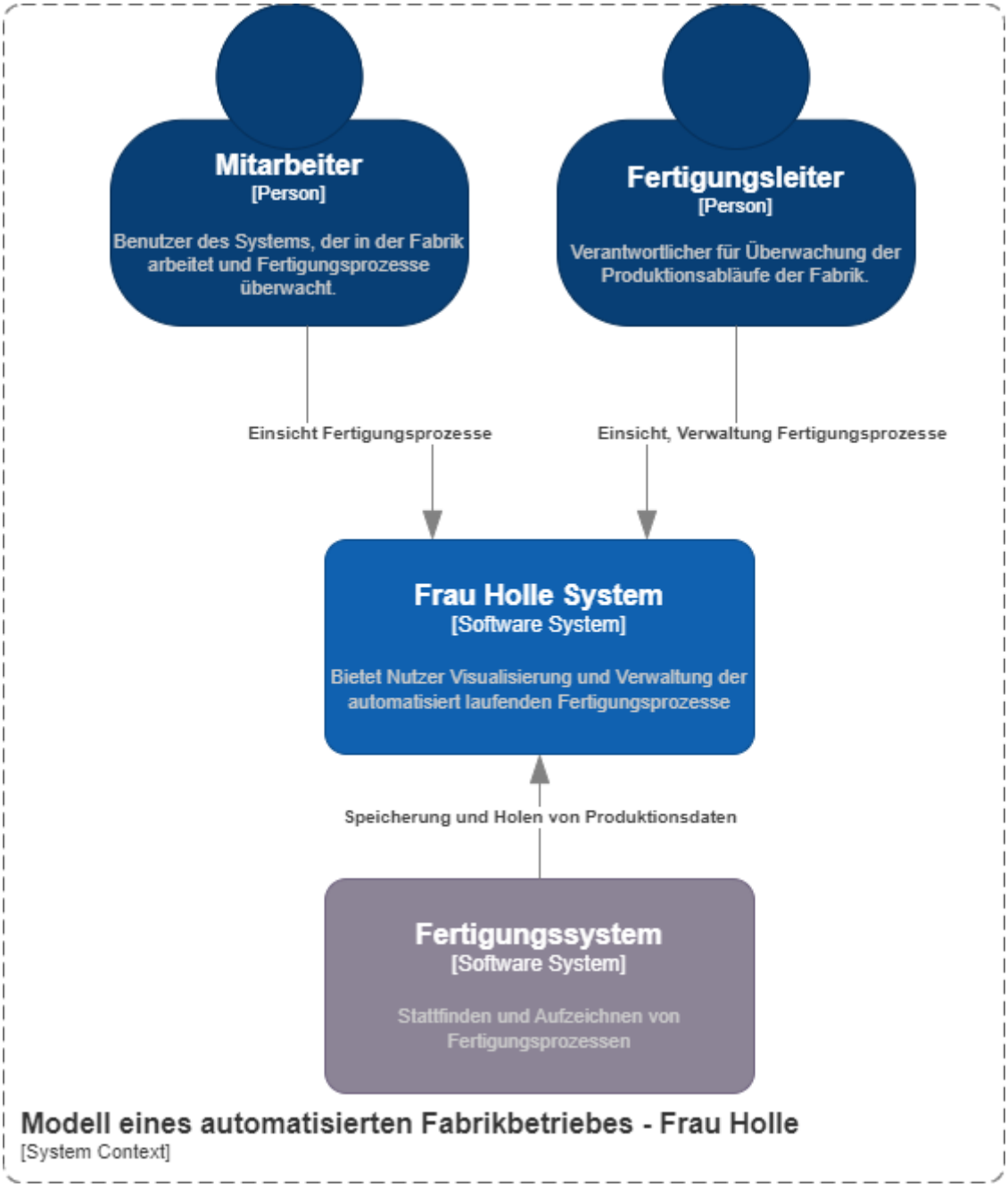


Abbildung 1. System-Kontext-Diagramm:

Das System-Kontext-Diagramm zeigt die Interaktion des Systems mit externen Systemen und Akteuren. Hier stellt es die Interaktion des Frau Holle Systems mit den Nutzern "Systemadministrator" und "Mitabeiter" dar. Außerdem wird die Interaktion mit einem externen Fertigungssystem dargestellt, in welchem die Fertigungsprozesse stattfinden.

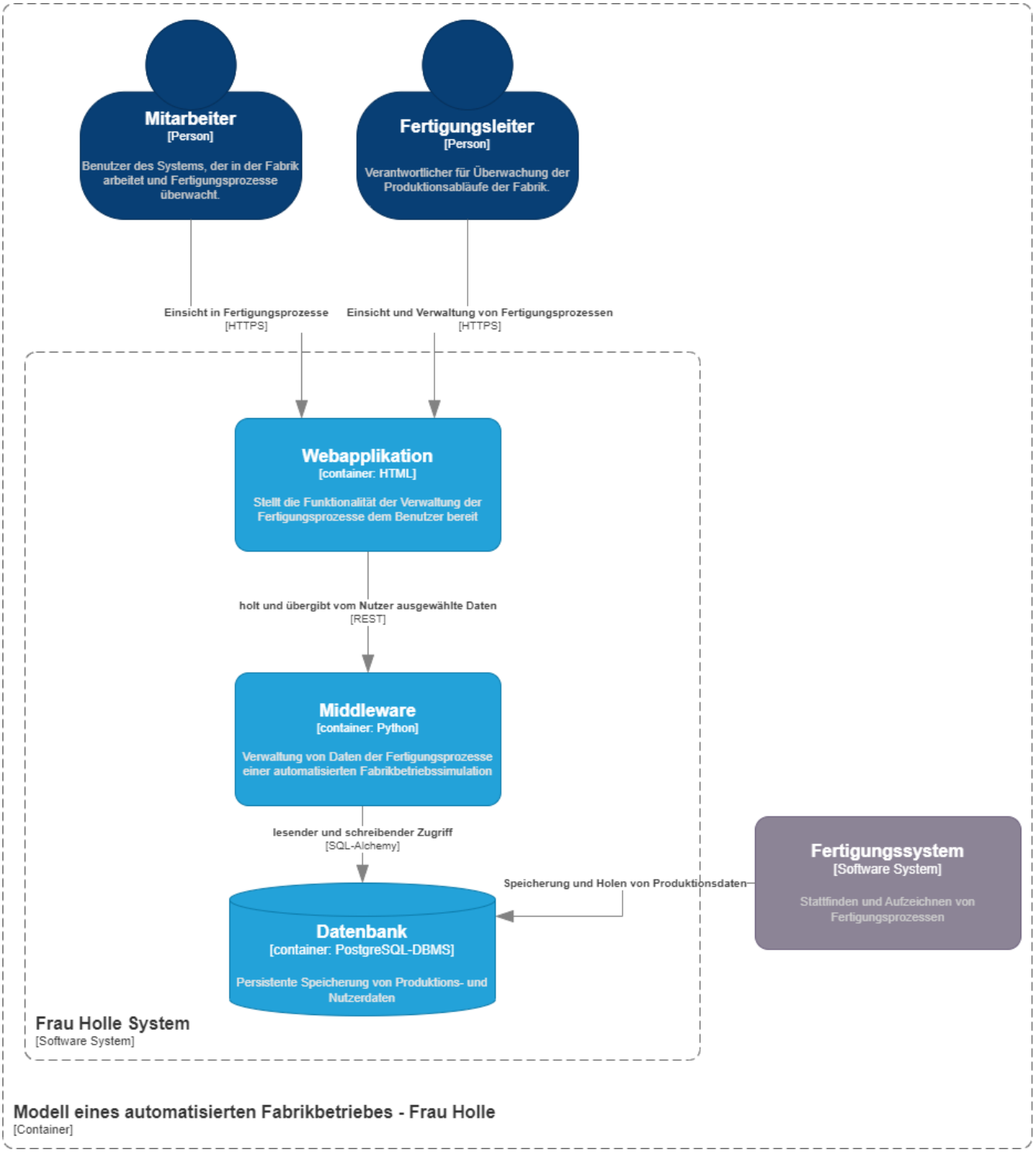


Abbildung 2. Container-Diagramm:

Das Container-Diagramm zeigt die Container, aus denen das System besteht und wie diese miteinander interagieren. Hier wird die Interaktion der Middleware mit der Benutzeroberfläche und der Datenbank dargestellt. Zusätzlich sind bereits die Schnittstellen zwischen den Containern dargestellt.

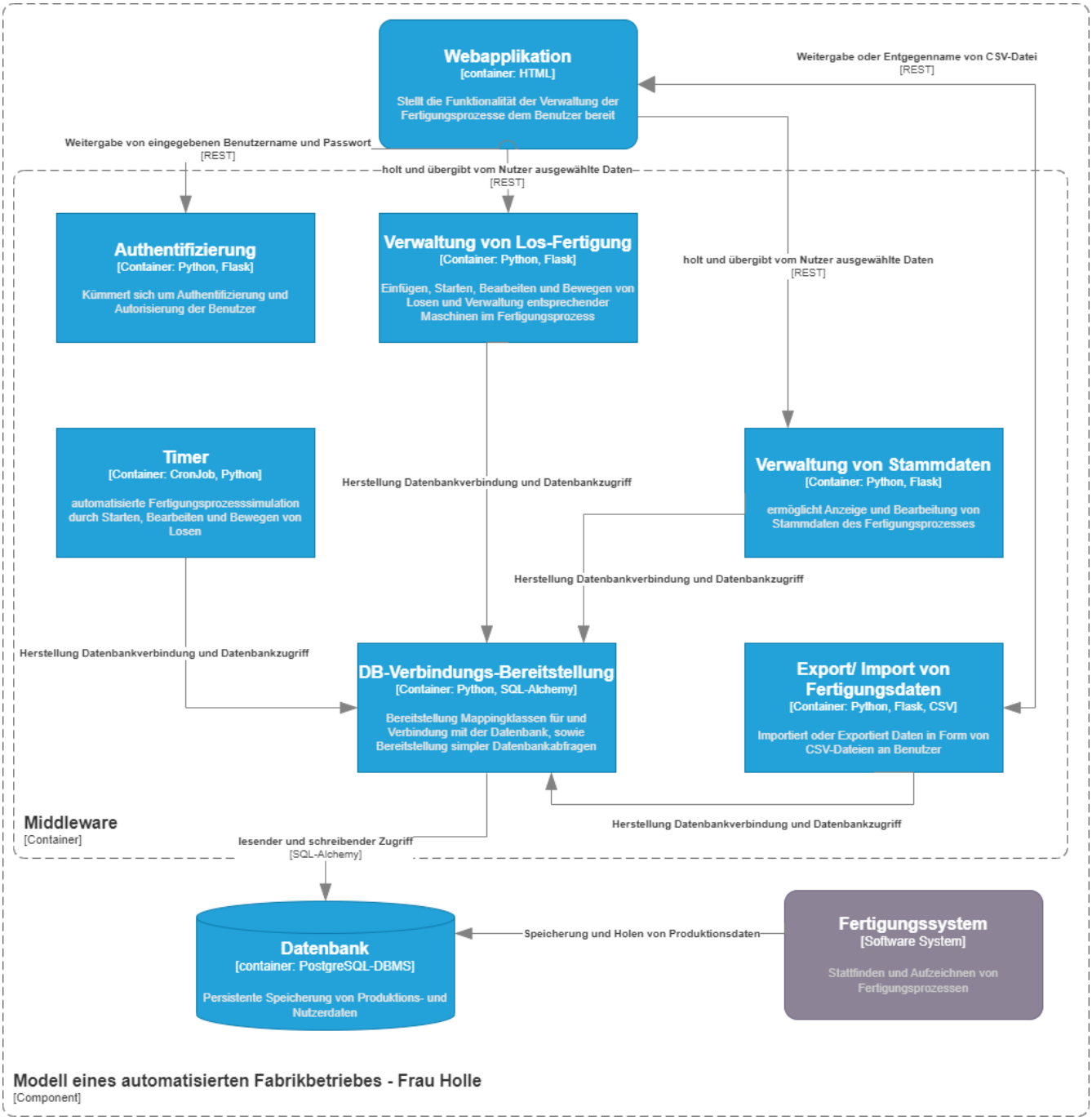
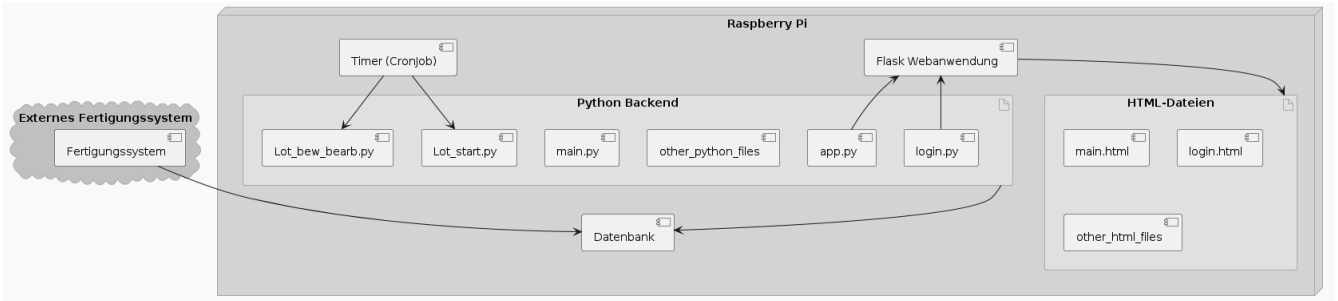


Abbildung 3. Component-Diagramm:

Das Component-Diagramm stellt die Interaktion der einzelnen Komponenten der Middleware dar. Außerdem ist die Art und Weise definiert, wie die Komponenten miteinander interagieren.

1.9.2. Physische Sicht (Betriebssicht)



Dieses Deployment-Diagramm zeigt die physische Verteilung der Softwarekomponenten auf den Hardwareknoten. Für das Frau Holle System wird damit dargestellt, wie die Flask-Anwendung einschließlich der Datenbank auf einem Raspberry Pi läuft. Die Flask-Anwendung interagiert mit HTML-Dateien und dem Python-Backend. Ein CronJob-Timer steuert die Ausführung von Python-Dateien für die automatische Fertigungssimulation. Das Konzept eines externen Fertigungssystems sollte direkt auf die zentrale Datenbank zugreifen.

1.9.3. Use cases

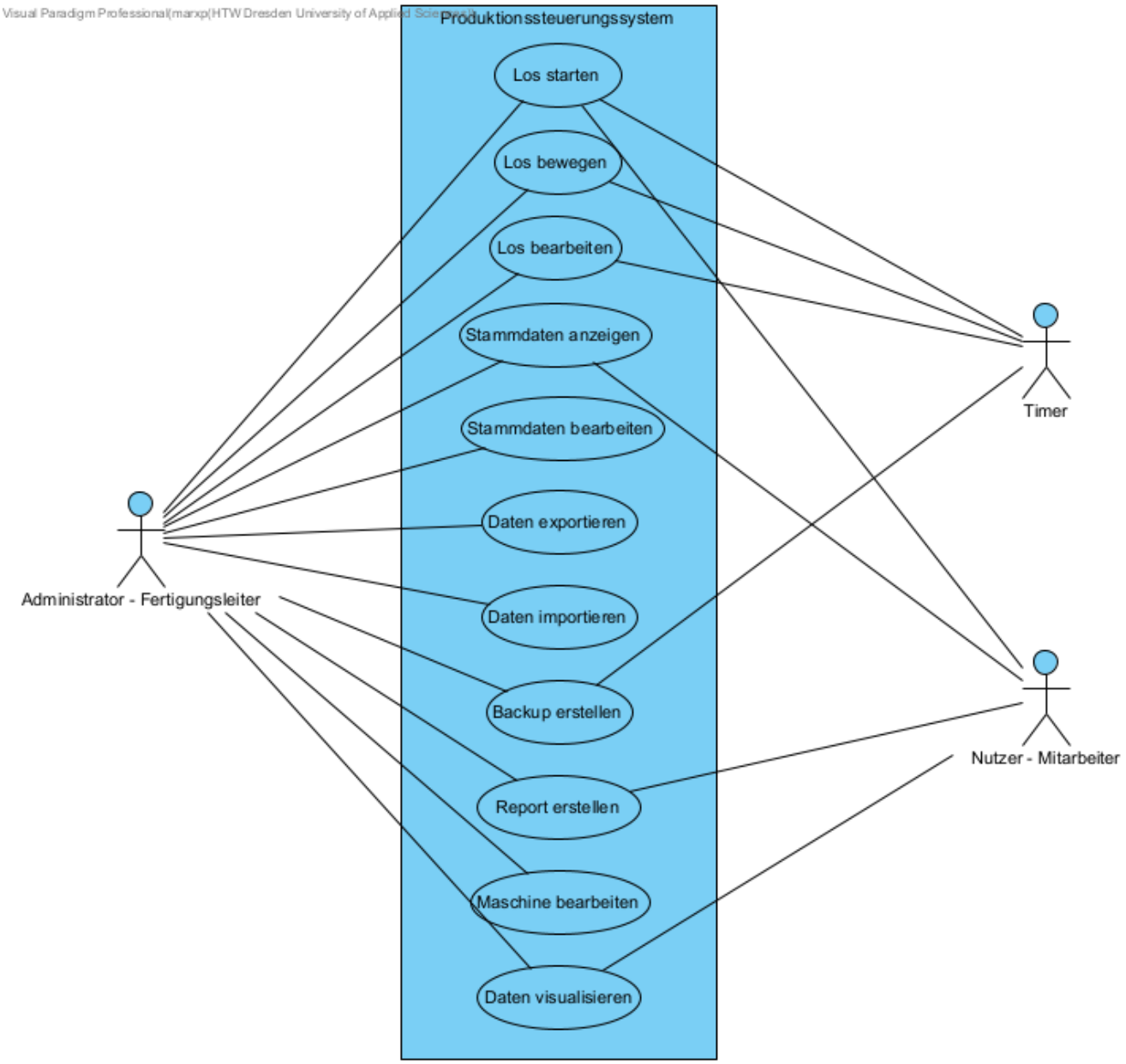


Abbildung 4. UseCaseDiagram

2. Softwaredokumentation

2.1. Allgemeine Informationen

Die Softwaredokumentation beinhaltet die Beschreibung aller Klassen bzw. aller Dokumente und die darin enthaltenen Funktionen. Zusätzlich dazu, sind in jedem Python-Dokument die entsprechenden Funktionen und Methoden kommentiert.

2.2. Frontend

Das Frontend ist in HTML, CSS und ein wenig in, in den HTML-Dateien eingebauten, JavaScript geschrieben. Es beinhaltet die Darstellung der Webseite und die Interaktion mit dem Benutzer. Betrieben wird das Frontend mit einem von Flask bereitgestellten Webserver, der die Anfragen an das Backend über die REST-API weiterleitet. Dynamische Inhalte in den HTML-Dateien werden mit Jinja2 gerendert.

2.2.1. Struktur/ Benutzungsablauf

Mittels folgender schematischer Darstellung wird die Struktur des Frontends dargestellt. Die einzelnen HTML-Dateien, die meist jeweils 1 Wireframe umsetzen, sind in der Reihenfolge ihrer möglichen Verwendung vom Benutzer aufgezeigt. Die Webapplikation startet immer im Login-Bereich und führt über die Registrierung dann zum Menübereich (Cockpit) des entsprechenden Nutzers. Von dort aus kann der Nutzer dann die verschiedenen Funktionen der Webapplikation nutzen.

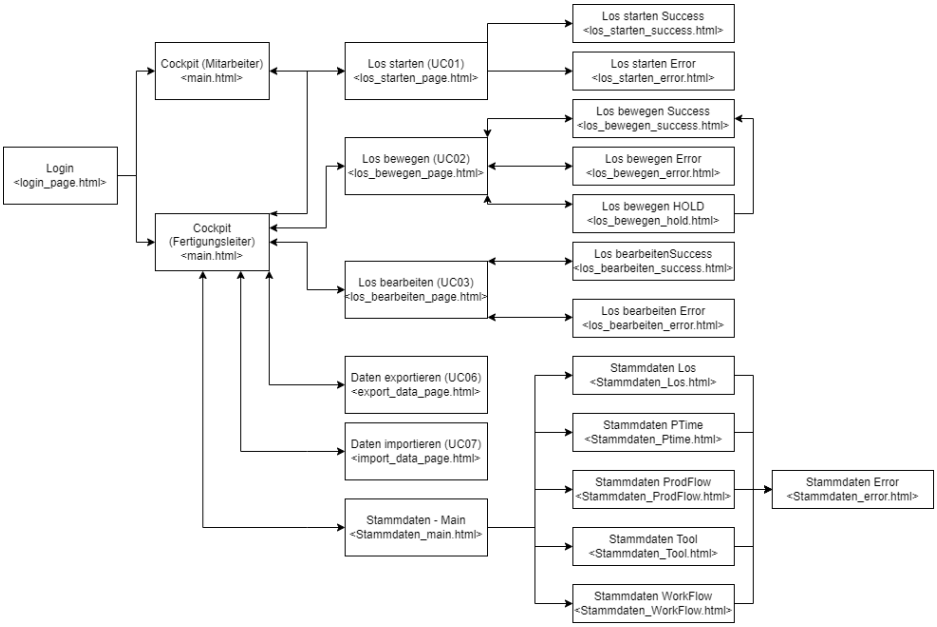


Abbildung 5. Zusammenhänge zwischen HTML-Seiten

2.2.2. Dynamische Inhalte mit Jinja2 eingebettet

Jinja2 ist ein Template-Engine für Python, die es ermöglicht, dynamische Inhalte in HTML-Dateien zu rendern. In den HTML-Dateien sind die Jinja2-Tags in doppelten geschweiften Klammern `{{ }}` eingebettet. Diese Tags können Variablen enthalten, die in der Python-Datei, die das HTML-

Template rendert, definiert sind. Dies wird beispielsweise für das filter- und sortierungsabhängige Füllen von Tabellen mit Daten aus der Datenbank, wie im folgenden Code-Beispiel aus der Datei `los_starten_page.html` aufgeführt, verwendet.

Beispiel: Jinja2-Tags in der HTML-Datei los_starten_page.html

```
    </tr>
</thead>
<tbody>
    {% for lot in lots %}
    <tr>
        <td>{{ lot.lotid }}</td>
        <td>{{ lot.state }}</td>
        <td>{{ lot.oper }}</td>
        <td>{{ lot.route }}</td>
        <td>{{ lot.product }}</td>
```

2.2.3. Funktionalitäten der HTML-Dateien

Die HTML-Dateien enthalten die Darstellung der Webseite und die Interaktion mit dem Benutzer. Zur Gewährleistung der Benutzerinteraktion wird die REST-API verwendet, um die Anfragen an das Backend zu senden. Die Funktionalitäten der wichtigsten HTML-Dateien sind in der folgenden Tabelle aufgeführt.

Tabelle 1. login_page.html

Funktionalität	Button-Name	bearb. Python-Datei	Beschreibung
Login	Login	login.py	Der Benutzer kann sich mittels Benutzername und Passwort einloggen.

Tabelle 2. main.html

Funktionalität	Button-Name	bearb. Python-Datei	Beschreibung
Log out	Log out	login.py	Der Benutzer kann sich ausloggen.
Wafer Informationen	-	-	Anzeige von verschiedenen Informationen über Wafer (WIP, HOLD, MVS/hr, COMP, FIN).
Tools Informationen	-	-	Anzeige von verschiedenen Informationen über Tools (Anzahl, IDLE, PROD, MAINT, DOWN).

Funktionalität	Button-Name	bearb. Python-Datei	Beschreibung
Los starten	starten	los_starten.py	Starten eines neuen Loses.
Los bearbeiten	bearbeiten	los_bearbeiten.py	Bearbeiten eines bestehenden Loses (nur für Fertigungsleiter sichtbar).
Los bewegen	bewegen	los_bewegen.py	Bewegen eines Loses (nur für Fertigungsleiter sichtbar).
Stammdaten	Stammdaten	Stammdaten.py	Einsicht und Anpassung der Stammdaten.
Daten exportieren	Daten exportieren	export_data.py	Export von Daten (nur für Fertigungsleiter sichtbar).
Daten importieren	Daten importieren	import_data.py	Import von Daten (nur für Fertigungsleiter sichtbar).

Tabelle 3. los_starten_page.html

Funktionalität	Button-Name	bearb. Python-Datei	Beschreibung
Zurückgehen	Back	- (per JavaScript)	Rückgängig machen der letzten Aktion.
Back to Main Page	Back to Main Page	-	Zurück zur Hauptseite, abhängig vom Benutzernamen.
Auftrag anlegen	Register Order	los_starten.py	Formular zur Erstellung eines neuen Auftrags abschicken.
Los manuell starten	Start Lot	los_starten.py	Formular zum manuellen Starten eines Loses abschicken.
Filtern von Losen	Filtern	los_starten.py	Möglichkeit, Los-Daten basierend auf verschiedenen Kriterien zu filtern.

Funktionalität	Button-Name	bearb. Python-Datei	Beschreibung
Sortieren von Anzeige der Lose	-	los_starten.py	Möglichkeit, die Anzeige der Lose basierend auf verschiedenen Kriterien zu sortieren.
Alle Lose anzeigen	-	los_starten.py	Anzeige aller Los-Daten in einer Tabelle.

Tabelle 4. los_bearbeiten_page.html

Funktionalität	Button-Name	bearb. Python-Datei	Beschreibung
Zurückgehen	Back	- (per JavaScript)	Rückgängig machen der letzten Aktion.
Back to Main Page	Back to Main Page	-	Zurück zur Hauptseite, abhängig vom Benutzernamen.
Los bearbeiten	Los bearbeiten	los_bearbeiten.py	Formular mit Angabe eines bestehenden Loses zur Bearbeitung abschicken.
Filtern von Losen	Filtern	los_bearbeiten.py	Möglichkeit, Los-Daten basierend auf verschiedenen Kriterien zu filtern.
Alle Lose anzeigen	-	los_bearbeiten.py	Anzeige aller Los-Daten in einer Tabelle.
Filtern von Tools	Filtern	los_bearbeiten.py	Möglichkeit, Tool-Daten basierend auf verschiedenen Kriterien zu filtern.
Alle Tools anzeigen	-	los_bearbeiten.py	Anzeige aller Tool-Daten in einer Tabelle.
Sortieren von Anzeige	-	los_bearbeiten.py	Möglichkeit, die Anzeige der Lose oder Tools basierend auf verschiedenen Kriterien zu sortieren.

Tabelle 5. los_bewegen_page.html

Funktionalität	Button-Name	bearb. Python-Datei	Beschreibung
Zurückgehen	Back	- (per JavaScript)	Rückgängig machen der letzten Aktion.

Funktionalität	Button-Name	bearb. Python-Datei	Beschreibung
Back to Main Page	Back to Main Page	-	Zurück zur Hauptseite, abhängig vom Benutzernamen.
Los bewegen	Move Lot	los_bewegen.py	Formular mit Angabe eines bestehenden Loses zum Bewegen abschicken.
Filtern von Losen	Filtern	los_bewegen.py	Möglichkeit, Los-Daten basierend auf verschiedenen Kriterien zu filtern.
Alle Lose anzeigen	-	los_bewegen.py	Anzeige aller Los-Daten in einer Tabelle.
Sortieren von Anzeige der Lose	-	los_bewegen.py	Möglichkeit, die Anzeige der Lose basierend auf verschiedenen Kriterien zu sortieren.

Tabelle 6. Stammdaten_main.html

Funktionalität	Button-Name	bearb. Python-Datei	Beschreibung
Zurückgehen	Back	- (per JavaScript)	Rückgängig machen der letzten Aktion.
Back to Main Page	Back to Main Page	-	Zurück zur Hauptseite, abhängig vom Benutzernamen.
Lot anzeigen	Lot	Stammdaten.py	Weiterleitung zur Seite für Los-Stammdaten.
Tool anzeigen	Tool	Stammdaten.py	Weiterleitung zur Seite für Tool-Stammdaten.
Product-Flow anzeigen	Product-Flow	Stammdaten.py	Weiterleitung zur Seite für Product-Flow-Stammdaten.
Work-Flow anzeigen	Work-Flow	Stammdaten.py	Weiterleitung zur Seite für Work-Flow-Stammdaten.
PTime anzeigen	PTime	Stammdaten.py	Weiterleitung zur Seite für PTime-Stammdaten.

Tabelle 7. Stammdaten_Los.html

Funktionalität	Button-Name	bearb. Python-Datei	Beschreibung
Zurückgehen	Back	- (per JavaScript)	Rückgängig machen der letzten Aktion.
Back to Main Page	Back to Main Page	-	Zurück zur Hauptseite, abhängig vom Benutzernamen.
Lot ID eingeben	select Lot ID	Stammdaten.py	Ermöglicht die Auswahl eines Lots zur Bearbeitung.
Lot-Daten bearbeiten	Edit Lot	Stammdaten.py	Ermöglicht die tatsächliche Bearbeitung der dann angezeigten Lot-Daten auf Basis der eingegebenen Lot-ID.
Lot-Bearbeitung abschließen	Submit	Stammdaten.py	Abschließen der Bearbeitung der Lot-Daten durch konsistentes Speichern der Änderungen.
Filtern nach Lot ID und Zustand	Filter	Stammdaten.py	Filtert die Tabelle basierend auf Lot ID und Zustand.
Sortieren der Tabelle	-	Stammdaten.py	Erlaubt das Sortieren der Tabelle nach verschiedenen Spalten.
Alle Lose anzeigen	-	Stammdaten.py	Anzeige aller Los-Daten in einer Tabelle.

Die Dateien der anderen HTML-Seiten sind potenziell analog zu den oben aufgeführten Dateien aufzubauen oder bereits aufgebaut.

Tabelle 8. import_data_page.html

Funktionalität	Button-Name	bearb. Python-Datei	Beschreibung
Zurückgehen	Back	- (per JavaScript)	Rückgängig machen der letzten Aktion.
Back to Main Page	Back to Main Page	-	Zurück zur Hauptseite, abhängig vom Benutzernamen.

Funktionalität	Button-Name	bearb. Python-Datei	Beschreibung
CSV-Datei importieren	Hinzufügen	import_data.py	Ermöglicht das Hochladen und Importieren einer CSV-Datei mit spezifischen Anforderungen.
Anzeige der Datenbankmeldungen	flash-messages-div-Element	-	Anzeige von Erfolgsmeldungen und Fehlermeldungen nach Importanforderung von Daten.
Entities hinzufügen	Hinzufügen	import_data.py	Fügt eine neue Entität zur Datenbank hinzu.
Lot hinzufügen	Hinzufügen	import_data.py	Fügt ein neues Lot zur Datenbank hinzu.
ProdFlow hinzufügen	Hinzufügen	import_data.py	Fügt einen neuen Produktionsfluss zur Datenbank hinzu.
PTime hinzufügen	Hinzufügen	import_data.py	Fügt eine neue Produktionszeit zur Datenbank hinzu.
Tool hinzufügen	Hinzufügen	import_data.py	Fügt ein neues Werkzeug zur Datenbank hinzu.
Workflow hinzufügen	Hinzufügen	import_data.py	Fügt einen neuen Arbeitsablauf zur Datenbank hinzu.

Tabelle 9. export_data_page.html

Funktionalität	Button-Name	bearb. Python-Datei	Beschreibung
Zurückgehen	Back	- (per JavaScript)	Rückgängig machen der letzten Aktion.
Back to Main Page	Back to Main Page	-	Zurück zur Hauptseite, abhängig vom Benutzernamen.
Filtern	Filtern	export_data.py	Ermöglicht das Filtern von Lots basierend auf Lot ID, Zustand und Sortierreihenfolge.
Select All	Select All	-	Markiert alle verfügbaren Lots für den Export.

Funktionalität	Button-Name	bearb. Python-Datei	Beschreibung
Export Lots to CSV	Export to CSV	export_data.py	Exportiert ausgewählte Lots in eine CSV-Datei.

2.2.4. CSS-Dateien und ihre Verwendung in den HTML-Dateien

Folgende CSS-Dateien werden von den HTML-Dateien verwendet, um die Darstellung der Webseite zu gestalten. Dabei wird ein für alle HTML-Dateien standardisiertes Design verwendet, um eine konsistente Benutzererfahrung zu gewährleisten. Folgende CSS-Dateien werden verwendet:

Tabelle 10. CSS-Dateien und ihre Verwendung in den HTML-Dateien

CSS-Datei	Verwendung	nutzende HTML-Dateien
`login.css`	Gestaltung der Login-Seite.	`login_page.html`
`main.css`	Gestaltung der Hauptseite.	`main.html`
`los_starten.css`	Gestaltung der Seite zum Starten von Losen.	`los_starten_page.html`
`los_bewegen.css`	Gestaltung der Seite zum Bewegen von Losen.	`los_bewegen_page.html`
`los_bewegen_hold.css`	Gestaltung der Seite, welche sich um die individuelle Behandlung von angehaltenen Losen kümmert.	`los_bewegen_hold.html`
`los_bearbeiten.css`	Gestaltung der Seite zum Bearbeiten von Losen.	`los_bearbeiten_page.html`
`los_error_success.css`	Gestaltung der Seite, welche die Erfolgsmeldungen und Fehlermeldungen anzeigt.	`los_bearbeiten_error.html`, `los_bearbeiten_success.html`,`los_bewegen_error.html`, `los_bewegen_success.html`, `los_starten_error.html`, `los_starten_success.html`
`Stammdaten_main.css`	Gestaltung der Stammdaten-Main-Seite.	`Stammdaten_main.html`
`Stammdaten_sites.css`	Gestaltung aller spezifischen Stammdaten-Tabellen-Seiten.	`Stammdaten_Los.html`, `Stammdaten_Tool.html`, `Stammdaten_ProdFlow.html`, `Stammdaten_WorkFlow.html`, `Stammdaten_PTime.html`
`port_data.css`	Gestaltung der Import- und Export-Seite.	`export_data_page.html`, `import_data_page.html`

2.3. Backend

2.3.1. Benutzerinteraktion mittels REST-API

Die Interaktion zwischen dem Frontend und dem Backend der Webapplikation wird durch eine REST-API ermöglicht. Diese API stellt verschiedene Endpunkte bereit, über die das Frontend Anfragen an das Backend senden kann. Die REST-API ist so gestaltet, dass sie die CRUD-Operationen (Create, Read, Update, Delete) für die verschiedenen Datenmodelle der Anwendung unterstützt. Die API verwendet das JSON-Format für den Datenaustausch und basiert auf dem HTTP-Protokoll.

Endpunkte der REST-API

Die REST-API umfasst mehrere Endpunkte, die jeweils spezifische Funktionen bereitstellen. Die folgenden Tabellen geben einen Überblick über die wichtigsten Endpunkte der REST-API, ihre Methoden und die entsprechenden Funktionen im Backend.

Tabelle 11. Authentifizierungs-Endpunkte

Endpunkt	HTTP-Methode	Beschreibung	Backend-Funktion
`/`	GET	Zeigt die Login-Seite an.	`login()`
`/api/login_check`	POST	Überprüft die Benutzerdaten und setzt die Sitzungsinformationen.	`login_check()`
`/api/logout`	GET	Löscht die Sitzungsinformationen und beendet die Sitzung.	`logout()`

Tabelle 12. Main-Endpunkte

Endpunkt	HTTP-Methode	Beschreibung	Backend-Funktion
`/main`	GET	Zeigt die Hauptseite für USER1 an, welche Daten über Maschinen und Lose anzeigt.	`main()`
`/main2`	GET	Zeigt die Hauptseite für USER2 an, welche Daten über Maschinen und Lose anzeigt.	`main2()`

Tabelle 13. Los-Starten-Endpunkte

Endpunkt	HTTP-Methode	Beschreibung	Backend-Funktion
`/main/los_starten`	GET	Zeigt die Seite zum Starten von Losen an und realisiert Filter- und Sortierungsoperationen .	`los_starten_page()`

Endpunkt	HTTP-Methode	Beschreibung	Backend-Funktion
`/main/los_starten`	POST	Erstellt neue Aufträge und Lose entsprechend der benutzerdefinierten Eingaben.	`los_starten()`
`/main/manuell_starten`	POST	Startet ein Los manuell durch den Benutzer.	`manuell_starten()`

Tabelle 14. Los-Bewegen-Endpunkte

Endpunkt	HTTP-Methode	Beschreibung	Backend-Funktion
`/main/los_bewegen`	GET	Zeigt die Seite zum Bewegen von Losen an und realisiert Filter- und Sortierungsoperationen.	`los_bewegen_page()`
`/main/los_bewegen`	POST	Bewegt ein Los mit benutzergegebener LotID zur nächsten Station und setzt entsprechend des Erfolges den neuen Zustand.	`los_bewegen()`

Tabelle 15. Los-Bearbeiten-Endpunkte

Endpunkt	HTTP-Methode	Beschreibung	Backend-Funktion
`/main/los_bearbeiten`	GET	Zeigt die Seite zum Bearbeiten von Losen an und realisiert Filter- und Sortierungsoperationen.	`los_bearbeiten_page()`
`/main/los_bearbeiten`	POST	Bearbeitet ein Los mit benutzergegebener LotID, indem es zur nächsten verfügbaren Maschine zugewiesen wird.	`moveLot()`

Tabelle 16. Stammdaten-Endpunkte

Endpunkt	HTTP-Methode	Beschreibung	Backend-Funktion
`/main/Stammdaten`	GET	Zeigt die Hauptseite der Stammdaten für USER1 an.	`Stammdaten_page()`
`/main/Stammdaten/Los`	GET	Zeigt die Stammdaten-Seite für Lose an, ermöglicht das Abrufen und Bearbeiten von Los-Daten.	`Stammdaten_Los_page()`
`/main/Stammdaten/Tool`	GET	Zeigt die Stammdaten-Seite für Werkzeuge an, ermöglicht das Abrufen und Bearbeiten von Werkzeug-Daten.	`Stammdaten_Tool_page()`
`/main/Stammdaten/ProdFlow`	GET	Zeigt die Stammdaten-Seite für Produktionsfluss an, ermöglicht das Abrufen und Bearbeiten von Produktionsfluss-Daten.	`Stammdaten_ProdFlow_page()`
`/main/Stammdaten/WorkFlow`	GET	Zeigt die Stammdaten-Seite für Workflow an, ermöglicht das Abrufen und Bearbeiten von Workflow-Daten.	`Stammdaten_WorkFlow_page()`
`/main/Stammdaten/PTime`	GET	Zeigt die Stammdaten-Seite für Produktionszeit an, ermöglicht das Abrufen und Bearbeiten von Produktionszeit-Daten.	`Stammdaten_PTime_page()`

Tabelle 17. Daten-Import-Endpunkte

Endpunkt	HTTP-Methode	Beschreibung	Backend-Funktion
`/import`	GET, POST	Ermöglicht den Import von CSV-Dateien.	`import_data()`
`/add_entity`	POST	Fügt eine neue Entität hinzu.	`add_entity()`
`/add_lot`	POST	Fügt ein neues Lot hinzu.	`add_lot()`
`/add_prodflow`	POST	Fügt einen neuen Produktionsfluss hinzu.	`add_prodflow()`

Endpunkt	HTTP-Methode	Beschreibung	Backend-Funktion
`/add_ptime`	POST	Fügt eine neue Produktionszeit hinzu.	`add_ptime()`
`/add_tool`	POST	Fügt ein neues Werkzeug hinzu.	`add_tool()`
`/add_workflow`	POST	Fügt einen neuen Workflow hinzu.	`add_workflow()`

Tabelle 18. Daten-Export-Endpunkte

Endpunkt	HTTP-Methode	Beschreibung	Backend-Funktion
`/main/export_data`	GET	Zeigt die Exportseite an, auf der Benutzer auswählen können, welche Daten sie exportieren möchten.	`export_data_page()`
`/main/export_data`	POST	Exportiert die ausgewählten Daten in eine CSV-Datei und sendet sie als Anhang zurück.	`export_to_csv()`

Implementierung der REST-API

Die REST-API ist in Flask implementiert, einem Mikro-Webframework für Python. Flask bietet einfache und flexible Möglichkeiten zur Erstellung von Webanwendungen und APIs. Die Endpunkte der API sind als Routen in den entsprechenden Python-Dateien definiert. Jeder Route ist einer spezifischen Funktion zugeordnet, die die spezifische Operation ausführt.

Listing 1. Beispiel: Definition eines API-Endpunkts in Flask in main.py

```
from flask import Blueprint, render_template, request, redirect, url_for, session
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
from mapper import Tool, Lot
from database_connection import get_database_url
from config import config
main_bp = Blueprint('main', __name__)

@main_bp.route('/main', methods=['GET'])
def main():
```

Fehlermeldungen und Statuscodes

Es kann ein 400 Bad Request-Code zurückgegeben werden, wenn die Eingabedaten ungültig sind, oder ein 404 Not Found-Code, wenn das angeforderte Los nicht existiert. Die Fehlermeldungen

werden als JSON-Objekte zurückgegeben, die den Fehlercode und eine entsprechende Nachricht enthalten.

Durch die Verwendung einer REST-API wird die Trennung von Frontend und Backend erleichtert, was eine klare Strukturierung und Wartbarkeit der Anwendung ermöglicht. Die REST-API stellt sicher, dass die verschiedenen Komponenten der Webapplikation effizient und zuverlässig miteinander kommunizieren können.

2.3.2. app.py und Blueprints

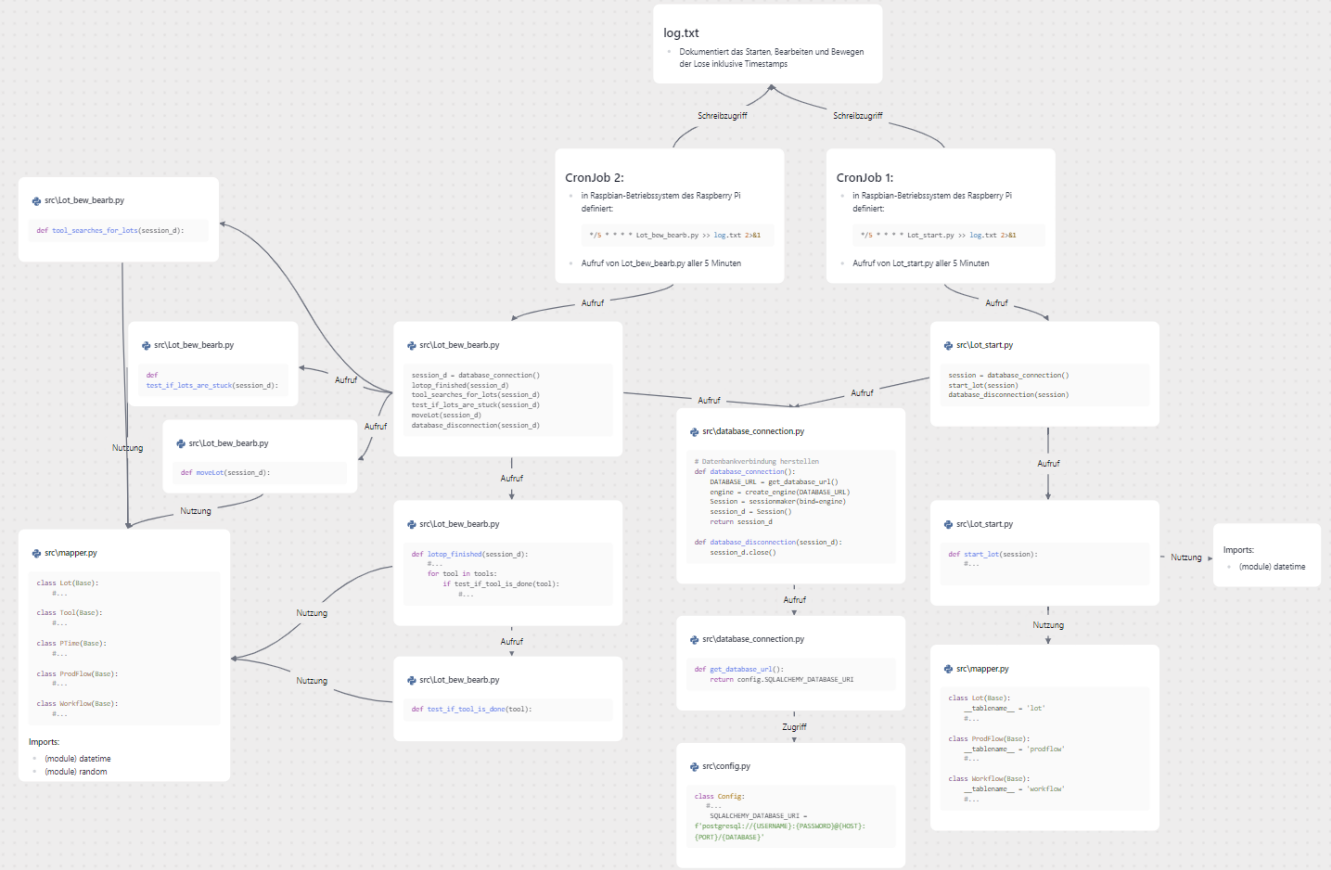
Die Datei `app.py` ist die Hauptdatei der Webapplikation und initialisiert die Flask-App. Hier werden die Blueprints für die verschiedenen Komponenten der Webapplikation registriert und die grundlegenden REST-API-Endpunkte definiert. Die Blueprints ermöglichen es, die verschiedenen Komponenten der Webapplikation zu strukturieren. Jedes Blueprint enthält die Routen und Funktionen für eine spezifische Komponente der Webapplikation. Die Blueprints werden in den Python-Dateien für die jeweiligen Komponenten definiert und in der Datei `app.py` registriert. Zum Start der Applikation wird auch diese Datei mittels `python app.py` ausgeführt.

Tabelle 19. Blueprints und ihre zugehörigen Python-Dateien

Blueprint	**Python-Datei**
login_bp	login.py
main_bp	main.py
los_starten_bp	los_starten.py
los_bearbeiten_bp	los_bearbeiten.py
los_bewegen_bp	los_bewegen.py
Stammdaten_bp	Stammdaten.py
import_data_bp	import_data.py
export_data_bp	export_data.py

2.3.3. Timer

Für die Implementierung des Timers wurde ein Code-Diagramm für die bessere Visualisierung und Nachvollziehbarkeit des Codes erstellt. Das Diagramm zeigt die Struktur des Timers und die Interaktion mit den anderen Komponenten des Systems.



Der Timer ist in Form zweier Cron-Jobs implementiert, welche in regelmäßigen Abständen die Fertigung der Lose überprüfen, diese in weitere Fertigungsschritte bewegen und bei Bedarf die Fertigung einzelner Lose startet. Der erste CronJob ist für das Überprüfen der Startdaten der Lose und das dementsprechende Starten dieser bei Erreichen des Startdatums zuständig. Der zweite CronJob überprüft die Fertigungsschritte der Lose und bewegt diese in den nächsten Fertigungsschritt, wenn der aktuelle Fertigungsschritt abgeschlossen ist. Außerdem prüft er für jede Maschine, ob diese noch in Betrieb ist und ordnet nach dem FIFO-Prinzip (First-In-First-Out) die wartende Lose entsprechend inaktiven Maschinen zu. Beide Cronjobs werden alle 5 Minuten vom Betriebssystem des Raspberry Pi ausgeführt. Entsprechend des Code-Diagramms, werden Logging-Informationen zu gestartete#n, bewegten und nun bearbeiteten Losen und die entsprechende Zeit in die Log-Datei `log.txt` geschrieben.

Timer: Lose starten

Das Starten von Losen, deren Startdatum erreicht wurde, wird durch einen CronJob, welcher die Datei `Lot_start.py` ausführt, realisiert. Der CronJob wird alle 5 Minuten ausgeführt. Die Funktion `start_lot()` in der Datei `Lot_start.py` überprüft die Startdaten der Lose und startet diese, wenn das Startdatum erreicht wurde. Zusätzlich wird lediglich eine Datenbanksession initialisiert und geschlossen.

Tabelle 20. Funktionen in Lot_start.py

Funktion	Beschreibung
start_lot()	Überprüft die Startdaten der Lose und startet diese, wenn das Startdatum erreicht wurde.

Timer: Lose bewegen und bearbeiten

Das Bewegen und Bearbeiten von Losen, die sich im Fertigungsprozess befinden, wird durch einen CronJob, welcher die Datei `Lot_bew_bearb.py` ausführt, realisiert. Der CronJob wird alle 5 Minuten ausgeführt. Die Funktionen dieser Datei überprüfen die Fertigungsschritte der Lose und bewegen diese in den nächsten Fertigungsschritt, wenn der aktuelle Fertigungsschritt abgeschlossen ist. Außerdem prüfen sie für jede Maschine, ob diese weiterhin noch in Betrieb ist und ordnen nach dem FIFO-Prinzip (First-In-First-Out) die wartenden Lose entsprechend inaktiven Maschinen zu.

Tabelle 21. Funktionen in Lot_bew_bearb.py

Funktion	Beschreibung	Parameter	Rückgabewert
test_if_tool_is_done()	Überprüft, ob eine übergebende Maschine im bearbeitenden Zustand ihr zu prozessierendes Los fertig bearbeitet hat.	tool: Tool	boolean
lotop_finished()	Für alle Lose, deren Maschine diese fertig bearbeitet hat, werden die Attribute entsprechend eines OP_COMP-Zustandes verändert.	session_d: DB-Session	-
tool_searches_for_lots()	Maschinen, welche sich im betriebsbereiten Zustand "IDLE" befinden, werden nach zu bearbeitenden Losen, welche als nächste Operation ein von der Maschine unterstützte Operation hat, durchsucht.	session_d: DB-Session	-
test_if_lots_are_stuck()	Prüfen, ob es Lose gibt, die in WAIT-Zustand sind, aber keine Maschine für nächste Operation und Route haben. Falls ja, setzt es diese auf den Zustand "HOLD".	session_d: DB-Session	-

Funktion	Beschreibung	Parameter	Rückgabewert
moveLot()	Schauen, ob ein Lot in OP-COMP-Zustand ist und versetzen des Lot auf nächsten Produktionsschritt.	session_d: DB-Session	-

CronJobs

Beide Cronjobs wurden auf dem Raspberry Pi eingerichtet. Beide CronJobs, welche die Dateien `Lot_start.py` und `Lot_bew_bearb.py` ausführen, werden alle 5 Minuten ausgeführt. Die CronJobs wurden mittels des Befehls `crontab -e` eingerichtet und die Ausführung der Dateien in den entsprechenden Intervallen festgelegt.

Listing 2. Beispiel: Einrichtung der CronJobs

```
*/5 * * * * /usr/bin/python ~/Desktop/i04-projekt-frau-holle/Lot_start.py >>
~/Desktop/i04-projekt-frau-holle/timer_log/log.txt 2>&1
*/5 * * * * /usr/bin/python ~/Desktop/i04-projekt-frau-holle/Lot_bew_bearb.py >>
~/Desktop/i04-projekt-frau-holle/timer_log/log.txt 2>&1
```

Angezeigt können die CronJobs mittels des Befehls `crontab -l` werden. Die Ausgaben der CronJobs werden in der Log-Datei `log.txt` gespeichert.

2.3.4. weitere Komponenten des Systems

Die Bündelung der Geschäftslogik des Timers und die Benutzerinteraktion mittels REST-API erfolgt in den Python-Dateien. So wurden die Aktionen des Timers und weitere administrative Aktionen manuell für den Administrator und teils auch für den Mitarbeiter ausführbar gemacht. Die Python-Dateien sind, gebündelt nach der Zugehörigkeit zu den einzelnen Komponenten, die im C4-Modell dargestellt sind, mit ihren Funktionen in den folgenden Tabellen aufgeführt.

Authentifizierung

Die Authentifizierung wird durch die Python-Datei `login.py` realisiert. Hierbei wird der Benutzername und das Passwort des Benutzers überprüft und bei erfolgreicher Authentifizierung wird als Sitzungsinformation der username gespeichert, welcher für die weitere Benutzerinteraktion benötigt wird. So wird in jeder weiteren Anfrage an das Backend der username überprüft, um den Benutzer zu authentifizieren. Damit wird sichergestellt, dass gewisse Funktionen nur vom autorisierten Fertigungsleiter ausgeführt werden können.

Tabelle 22. Funktionen in login.py

Funktion	Beschreibung	Parameter	Rückgabewert
login()	Zeigt die Login-Seite `login_page.html`.	-	HTML-Seite mit dem Login-Formular

Funktion	Beschreibung	Parameter	Rückgabewert
login_check()	Überprüft die eingegebenen Benutzerdaten und setzt bei Korrektheit die Sitzungsinformationen.	-	JSON-Antwort mit Status und Nachricht des Validierungsergebnisses
logout()	Löscht die Sitzungsinformationen und leitet den Benutzer auf die Login-Seite zurück.	-	HTML-Seite mit dem dem Login-Formular

Datenbankzugriff

Der Zugriff auf die Datenbank erfolgt über die ORM-Bibliothek SQLAlchemy. Diese ermöglicht es, die Datenbank als Objekte zu repräsentieren und somit die Datenbankabfragen in Python-Code zu schreiben. Das Mapping der Datenbanktabellen auf Python-Klassen erfolgt mittels der Datei `mapper.py`. Hierbei werden entsprechend des Klassendiagramms die Tabellen der Datenbank auf Python-Klassen abgebildet. Die Python-Klassen enthalten die Attribute, die dann den Spalten der Datenbanktabellen entsprechen.

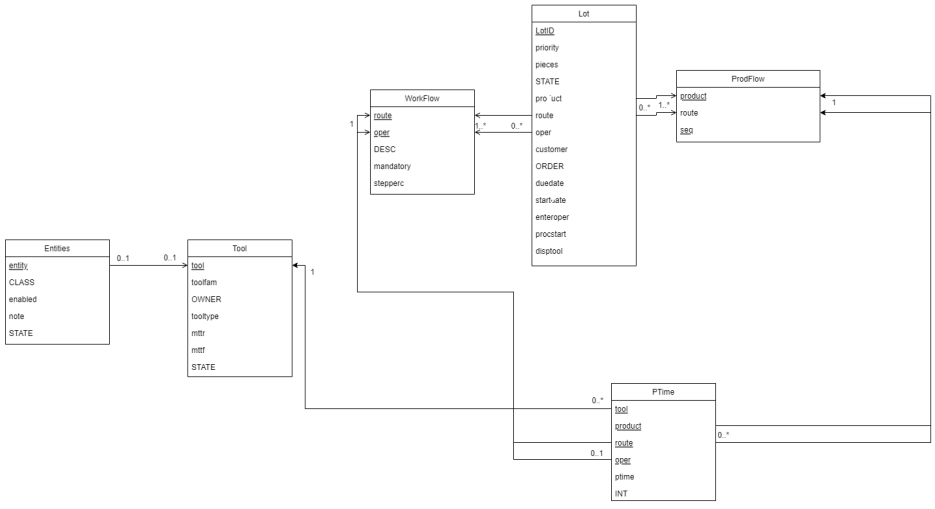


Abbildung 6. Klassendiagramm

Um auf die Datenbank zuzugreifen, wird in den Python-Dateien die Datenbank-Session `session` benötigt. Diese kann mittels der Funktionen in der Datei `database_connection.py` initialisiert und geschlossen werden.

Tabelle 23. Funktionen in database_connection.py

Funktion	Beschreibung	Parameter	Rückgabewert
get_database_url()	Gibt die Datenbank URL mit Config-Informationen zurück	-	URL für den Datenbankzugriff
database_connection()	Initialisiert die Datenbank-Session.	-	Datenbank-Session
database_disconnection()	Schließt die Datenbank-Session.	session: DB-Session	-

Mittels Import der jeweiligen Klassen und Funktionen aus den Dateien `mapper.py` und `database_connection.py` kann in jeder Python-Datei der Zugriff auf die Datenbank gewährleistet werden. Aufgrund der repetitiven Verwendung einiger einfacher Datenbankabfragen wurden diese in der Datei `database_accesses.py` zusammengefasst. Diese Funktionen können dann in den Python-Dateien importiert und verwendet werden. Beispielsweise wird in der Python-Datei `Stammdaten.py` die externe Funktion `get_all_lots()` verwendet, um alle Lose aus der Datenbank zu erhalten.

Verwaltung von Los-Fertigung

Die manuelle Verwaltung der Los-Fertigung durch den Benutzer wird durch die Python-Dateien `Los_starten.py`, `Los_bewegen.py` und `Los_bearbeiten.py` realisiert. Diese Dateien setzen die Funktionen des Timers als manuelle Aktionen um. So kann der Benutzer neue Lose starten, bestehende Lose bearbeiten und bewegen. Die Funktionen in diesen Dateien sind so implementiert, dass sie die entsprechenden Aktionen, analog zu den Timer-Aktionen, auf die Lose oder Maschinen in der Datenbank ausführen. Um die entsprechenden Aktionen anzuregen, werden die Funktionen in den Python-Dateien mittels der REST-API aufgerufen (siehe Endpunkte REST-API). Außerdem sind weitere Funktionalitäten implementiert, welche das Sortieren und das Filtern der angezeigten Lose und teilweise auch Tools ermöglichen. Dies ist insbesondere relevant, um die Übersicht über die Fertigung zu behalten.

Die Python-Datei `Los_starten.py` implementiert sowohl die Erstellung neuer Lose und das manuelle Starten bereits vorhandener Lose, welche nicht bereits schon zuvor (z.B. durch den Timer) gestartet wurden. Die wichtigsten Funktionen sind folgend aufgeführt.

Tabelle 24. Funktionen in Los_starten.py

Funktion	Beschreibung	Parameter	Rückgabewert
check_user_session()	Überprüft die Sitzungsinformationen und damit die Berechtigung des Benutzers.	-	boolean
los_starten_page()	Zeigt die Seite zum Starten von Losen an und kümmert sich ggf. um Sortierung und Filterung der angezeigten Lose.	-	Template oder "Unauthorized", 401

Funktion	Beschreibung	Parameter	Rückgabewert
los_starten()	Erstellt neue Aufträge und Lose entsprechend der benutzerdefinierten Eingaben.	-	Template oder "Unauthorized", 401
manuell_starten()	Startet ein Los manuell durch den Benutzer.	-	Template oder "Unauthorized", 401
generateLotID()	Generiert eine eindeutige Lot-ID für ein neues Los.	`create_lot_ids:` Liste an in dieser Session bereits generierten LotIDs, `customer:` String von Auftraggeber, `order:` String von Bestellungsname	string (mit der generierten Lot-ID)
get_sort_order()	Gibt die Sortierreihenfolge und -attribute für die Lose zurück.	`model:` Mappingklasse aus mapper.py , `sort_type:` String von Namen des gewählten Elements aus der sort-Auswahlliste	sqlalchemy.sql.elements.ColumnElement als Sortierattribut

Die Python-Datei `Los_bewegen.py` implementiert die Funktionen, um fertig bearbeitete Lose in ihren nächsten Produktionsschritt zu bewegen. Die wichtigsten Funktionen sind folgend aufgeführt.

Tabelle 25. Funktionen in Los_bewegen.py

Funktion	Beschreibung	Parameter	Rückgabewert
check_user_session()	Überprüft die Sitzungsinformationen und damit die Berechtigung des Benutzers.	-	boolean
los_bewegen_page()	Zeigt die Seite zum Bewegen von Losen an und kümmert sich ggf. um Sortierung und Filterung der angezeigten Lose.	-	Template oder "Unauthorized", 401

Funktion	Beschreibung	Parameter	Rückgabewert
los_bewegen()	Bewegt ein Los mit benutzergegebener LotID zur nächsten Station und setzt entsprechend des Erfolges den neuen Zustand. Außerdem implementiert es die Reaktion auf die manuelle Zustandszuordnung von SCRAP oder OP_COMP zu einem Los, welches sich im Zustand HOLD befindet.	-	Template oder "Unauthorized", 401
move_lot_to_next_station()	Bewegt ein Los mit benutzergegebenem Lot zur nächsten Operation und setzt entsprechend des Erfolges den neuen Zustand.	`lot:` Lot-Objekt, für das Lot, das beweget werden soll, `db_session:` sqlalchemy.orm.session.Session als Datenbankverbindung	integer (0 = Erfolgreich bewegt, 1 = Erfolgreich Produktionsfluss abgeschlossen, sonst = Fehler)
get_sort_order()	Gibt die Sortierreihenfolge und -attribute für die Lose zurück.	`model:` Mappingklasse aus mapper.py , `sort_type:` String von Namen des gewählten Elements aus der sort-Auswahlliste	sqlalchemy.sql.elements.ColumnElement als Sortierattribut

Die Python-Datei `Los_bearbeiten.py` implementiert die Funktionen, um Lose, welche sich im Fertigungsprozess gerade im (auf eine Maschine) wartenden Zustand befinden, auf eine eventuell verfügbare Maschine zu setzen und somit ihre nächste Operation im Fertigungsprozess zu starten. Die wichtigsten Funktionen sind folgend aufgeführt.

Tabelle 26. Funktionen in Los_bearbeiten.py

Funktion	Beschreibung	Parameter	Rückgabewert
los_bearbeiten_page()	Zeigt die Seite zum Bearbeiten von Losen an und kümmert sich ggf. um Sortierung und Filterung der angezeigten Lose.	-	Template oder "Unauthorized", 401

Funktion	Beschreibung	Parameter	Rückgabewert
moveLot()	Ordnet einem Los mit per REST-API übergebener Los-ID, falls es im WAIT-Zustand ist, eine verfügbare Maschine zu und setzt es auf den Zustand "PROC".	-	Template oder "Unauthorized", 401
get_sort_order()	Gibt die Sortierreihenfolge und -attribute für die Lose und Tools zurück.	`model:` Mappingklasse aus mapper.py , `sort_type:` String von Namen des gewählten Elements aus der sort-Auswahlliste	sqlalchemy.sql.elements.ColumnElement als Sortierattribut

Verwaltung von Stammdaten

Die Verwaltung der Stammdaten, wie Lose, Tools, ProductionFlows, Workflows und Produktionszeiten, wird durch die Python-Datei `Stammdaten.py` realisiert. Diese Datei ermöglicht es mit den spezifisch dazugehörenden HTML- und CSS-Dateien, dass der Benutzer die Stammdaten verwalten kann. Dazu gehört, dass er die Daten in einer Tabelle angezeigt bekommt und die Möglichkeit hat, die Daten zu einer spezifischen ID zu bearbeiten. Die wichtigsten Funktionen aus dieser Datei sind folgend aufgeführt.

Tabelle 27. Funktionen in Stammdaten.py

Funktion	Beschreibung	Parameter	Rückgabewert
check_user_session()	Überprüft die Berechtigung des Benutzers für das Anzeigen der Daten.	-	boolean: True wenn angemeldet, sonst False
check_user_session_user1()	Überprüft die Berechtigung des Benutzers für das Bearbeiten der Daten (nur user1).	-	boolean: True wenn USER1 angemeldet, sonst False
Stammdate_page()	Rendert die Hauptseite der Stammdaten, wenn der Benutzer angemeldet ist.	-	Template oder "Unauthorized", 401
Stammdaten_Los_page()	Zeigt die Stammdaten-Seite für Lose an, ermöglicht das Abrufen, Filtern und Bearbeiten von Losen.	-	Template oder "Unauthorized", 401

Funktion	Beschreibung	Parameter	Rückgabewert
get_args_right()	Holt und validiert einen Request-Parameter.	id (String): Parametername	Parameterwert oder None
get_timestamp_args_right()	Holt und validiert einen Timestamp-Parameter und formatiert ihn.	id (String): Parametername	Formatierter Timestamp oder None

Alle weiteren Funktionen Stammdaten_X_page() für X={Tool,ProdFlow,WorkFlow,PTime} sind noch nicht final umgesetzt, werden aber vor Allem mit ihrer Funktionalität und ihrem Rückgabewert analog zur Stammdaten_Los_page() implementiert.

Export/Import von Fertigungsdaten

Die Export- und Importfunktionen für die Fertigungsdaten werden durch die Python-Dateien `export_data.py` und `import_data.py` realisiert. Diese Dateien ermöglichen es, dass der Benutzer die Fertigungsdaten in eine CSV-Datei exportieren und aus einer CSV-Datei importieren kann. Die wichtigsten Funktionen und Funktionalitäten aus beiden Datei sind folgend aufgeführt.

Tabelle 28. Funktionen in export_data.py

Funktion	Beschreibung	Parameter	Rückgabewert
check_user_session()	Überprüft, ob der Benutzer als `USER1` angemeldet ist.	-	boolean: `True` wenn `USER1` angemeldet, sonst `False`
export_data_page()	Rendert die Export-Daten-Seite und filtert die Lose basierend auf den angegebenen Parametern	-	Template oder "Unauthorized", 401
get_sort_order()	Gibt die Sortierreihenfolge und -attribute für die Lose und Tools zurück.	`model:` Mappingklasse aus mapper.py , `sort_type:` String von Namen des gewählten Elements aus der sort-Auswahlliste	sqlalchemy.sql.elements.ColumnElement als Sortierattribut
export_to_csv()	Exportiert die ausgewählten Lose in eine CSV-Datei und sendet diese als Anhang.	-	CSV-Datei als Anhang

Tabelle 29. Funktionen in import_data.py

Funktion	Beschreibung	Parameter	Rückgabewert
check_user_session()	Überprüft, ob der Benutzer als `USER1` angemeldet ist.	-	Boolean: `True` wenn `USER1` angemeldet, sonst `False`
allowed_file()	Überprüft, ob die Datei eine erlaubte CSV-Datei ist.	`filename:` Name der Datei	Boolean: `True` wenn die Datei erlaubt ist, sonst `False`
import_data()	Handhabt das Hochladen und Importieren von CSV-Dateien, rendert die Import-Daten-Seite.	-	Template oder "Unauthorized", 401
import_from_csv()	Importiert Daten aus einer CSV-Datei in die entsprechende Datenbanktabelle.	`csv_data:` CSV-Daten als String, `filename:` Name der Datei	String: Anzahl der erfolgreich hinzugefügten Einträge oder Fehlermeldung
add_instance()	Fügt eine Instanz der angegebenen Modellklasse mit den angegebenen Daten zur Datenbank hinzu.	`model_class:` Modellklasse, die hinzugefügt werden soll, `data:` Daten als Dictionary	None
add_entity()	Fügt eine `Entities` -Instanz zur Datenbank hinzu.	-	Redirect zur Import-Daten-Seite

Alle anderen `add_X`-Funktionen für X={Lot,ProdFlow,PTime,Tool,WorkFlow} sind final umgesetzt und analog zur `add_entity`-Funktion implementiert.

Entities-Verwaltung (Prototyp)

Die Datei `entities.py`, sowie die dazugehörige HTML-Datei `entities_page.html` sind lediglich ein Prototyp für die Benutzung der Schnittstellen dieses Systems. Hierbei wird die Möglichkeit gegeben, neue Entitäten zu erstellen, zu bearbeiten und zu löschen. Diese Funktionalität ist nicht im finalen System implementiert, sondern dient lediglich als Beispiel für die Benutzung der Schnittstellen.

Konzept: Backup erstellen

Das Konzept für das Erstellen eines Backups der Datenbank ist in der Datei `backup.py`, welche das Shellskript `backup.sh` aufruft. Hierbei wird ein Backup der Datenbank erstellt und in einem Backup-Ordner gespeichert. Das Backup wird als SQL-Datei gespeichert und enthält alle Tabellen und Daten der Datenbank. Das Backup kann bei Bedarf wiederhergestellt werden, um die Datenbank in einen früheren Zustand zu versetzen. Diese Funktionalität ist nicht im aktuell auf dem Raspberry Pi laufenden System implementiert, sondern dient als Konzept für die Sicherung der Datenbank. Die aktuelle Implementierung erstellt zwar bereits ein Backup der Datenbank, jedoch wurde diese nicht ausreichend genug getestet und wird bisher auch noch nicht durch einen CronJob regelmäßig ausgeführt. Die System-Version mit der Backup-Funktionalität ist in einem separaten Branch `backup`

im Repository auf GitHub verfügbar. Außerdem liegt diese Version auch auf dem Raspberry Pi in einem separaten Verzeichnis und ist unter `~/Dokumente/i04-projekt-frau-holle-with-backup` zu finden.

2.3.5. Config-Datei

Die Konfigurationsdatei `config.py` beinhaltet alle Konfigurationen, die für die Webapplikation benötigt werden. Hierzu zählen beispielsweise die Informationen für den Datenbankzugriff, die URL für den Webserver oder der Applikationsname. Die Konfigurationsdatei wird in den Python-Dateien mittels `from config import Config` importiert und die Konfigurationen können dann über die Klasse `Config` abgerufen werden.

2.4. Datenbank

Die Datenbank ist in PostgreSQL implementiert und enthält die Tabellen für die Maschinen, Lose und andere Informationen für die Fertigung der Lose. Die Tabellen sind entsprechend des folgend aufgeführten Entity-Relationship-Modells aufgebaut und enthalten die notwendigen Attribute für die Speicherung der Daten. Die Datenbank mit ihren Relationen wird mittels der ORM-Bibliothek SQL-Alchemy in Python repräsentiert, wodurch die Datenbankabfragen in Python-Code geschrieben werden.

2.4.1. Entity-Relationship-Modell

Das Entity-Relationship-Modell zeigt die Beziehungen zwischen den verschiedenen Entitäten der auf dem Raspberry Pi liegenden Datenbank, welche die Maschinen, Lose, Maschinen, Produktionsflüsse, Workflows und Produktionszeiten beschreiben.

